Tarea 2: Path Tracing con OpenMP y Numba

Informe local generado a partir de `results/full_local_/results.csv` y `hardware.json`. No se encontraron resultados de un segundo computador en el repositorio; esa comparacion queda pendiente y no se inventan mediciones.

Declaracion de uso de IA

Se uso un asistente de IA para revisar la pauta, implementar las versiones OpenMP y Numba, automatizar experimentos, revisar la cobertura de resultados y generar este informe. Los resultados numericos provienen de ejecuciones locales registradas en `results.csv`.

Entorno de ejecucion

Dato	Valor
Procesador	Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
Cores fisicos	6
Threads logicos	12
Threads por core	2
Sistema	Linux-5.15.167.4-microsoft-standard-WSL2-x86_64-with-glibc2.39
Python	3.12.3

Cobertura de resultados

La corrida usada corresponde a la matriz completa local. Cada punto fue medido con wall-clock time y la corrida quedo registrada con `--reps 1`.

Bloque	Filas	Cobertura
Serial	12	serial_rt y serial_pt para 2 escenas x 3 configs
Scheduling OpenMP	88	11 estrategias/chunks x 4 threads x 2 escenas
Comparacion OpenMP	24	2 escenas x 3 configs x 4 threads
Comparacion Numba	24	2 escenas x 3 configs x 4 threads

Configuraciones

Config.	Resolucion	S ray tracing	N path tracing
small	400 x 300	2	32
medium	800 x 600	4	128
large	1600 x 1200	4	256

Implementacion

`omp_pt.cpp` paraleliza el loop de pixeles con `#pragma omp parallel for`. `omp_pt_sched.cpp` usa `schedule(runtime)` y `omp_set_schedule` para probar `static`, `dynamic` y `guided` con distintos chunks. Cada pixel es independiente y escribe una posicion unica del arreglo de salida, por lo que la paralelizacion sigue el patron map.

`numba_pt.py` implementa la misma idea con `@njit(parallel=True)` y `prange`. El script hace un warm-up minimo antes de medir para que el tiempo reportado no incluya la compilacion JIT inicial de Numba.

Resultados seriales

Escena	Config.	serial.cpp (s)	serial_pt.cpp (s)
scene.txt	small	0.083	4.075
scene.txt	medium	1.379	64.690
scene.txt	large	5.658	533.884
scene_many.txt	small	0.220	6.177
scene_many.txt	medium	3.484	98.010
scene_many.txt	large	13.775	782.014

El crecimiento de `serial_pt.cpp` fue:

Escena	small -> medium	medium -> large
scene.txt	15.87x	8.25x
scene_many.txt	15.87x	7.98x

Al pasar de `small` a `medium` se cuadruplica la resolucion y tambien aumenta N de 32 a 128, por lo que el costo esperado del path tracing crece cerca de 16x. Los resultados locales fueron 15.87x en ambas escenas, consistente con el modelo O(WHNDK). Al pasar de `medium` a `large`, la resolucion vuelve a crecer 4x y N se duplica de 128 a 256; el crecimiento esperado es cercano a 8x, y las mediciones fueron 8.25x para `scene.txt` y 7.98x para `scene_many.txt`.

Barrido de scheduling OpenMP

Escena	Mejor promedio	Tiempo prom. (s)
scene.txt	guided(8)	34.511
scene_many.txt	dynamic(128)	53.780

Escena	p=1	p=2	p=4	p=8
scene.txt	static(32), 64.336s	guided(8), 37.323s	static(1), 21.527s	dynamic(1), 14.067s
scene_many.txt	dynamic(128), 99.200s	guided(32), 57.949s	static(1), 34.195s	dynamic(8), 22.673s

No hubo una estrategia unica que dominara todos los puntos. En `scene.txt`, el mejor promedio global fue `guided(8)`, aunque por thread especifico aparecen casos donde `static` o `dynamic` ganan. En `scene_many.txt`, el mejor promedio global fue `dynamic(128)`. La diferencia se explica por la irregularidad del trabajo por pixel: algunos rayos terminan rapido y otros hacen mas rebotes, sombras e intersecciones. La escena con 40 esferas aumenta el costo K de cada interseccion y vuelve mas importante balancear la carga, pero chunks demasiado finos tambien agregan overhead de planificacion.

Comparacion OpenMP vs Numba

Para cada punto se usa como referencia el tiempo `serial_pt` de la misma escena y configuracion. Se calcula $S(p) = T_s/T(p)$ y $E(p) = S(p)/p$.

scene.txt - small

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	4.181	0.97	0.97	3.579	1.14	1.14	Numba
2	2.406	1.69	0.85	1.994	2.04	1.02	Numba
4	1.346	3.03	0.76	1.142	3.57	0.89	Numba
8	0.765	5.32	0.67	0.659	6.18	0.77	Numba

scene.txt - medium

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	70.217	0.92	0.92	51.965	1.24	1.24	Numba
2	39.678	1.63	0.82	32.416	2.00	1.00	Numba
4	23.280	2.78	0.69	18.847	3.43	0.86	Numba
8	15.464	4.18	0.52	12.602	5.13	0.64	Numba

scene.txt - large

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	581.278	0.92	0.92	399.574	1.34	1.34	Numba
2	302.865	1.76	0.88	261.006	2.05	1.02	Numba
4	175.561	3.04	0.76	159.330	3.35	0.84	Numba
8	112.857	4.73	0.59	102.854	5.19	0.65	Numba

scene_many.txt - small

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	6.308	0.98	0.98	5.488	1.13	1.13	Numba
2	3.659	1.69	0.84	3.482	1.77	0.89	Numba
4	2.227	2.77	0.69	2.049	3.02	0.75	Numba
8	1.251	4.94	0.62	1.298	4.76	0.59	OpenMP

scene_many.txt - medium

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	101.456	0.97	0.97	87.361	1.12	1.12	Numba
2	58.990	1.66	0.83	56.068	1.75	0.87	Numba
4	34.330	2.85	0.71	34.581	2.83	0.71	OpenMP
8	22.855	4.29	0.54	23.217	4.22	0.53	OpenMP

scene_many.txt - large

p	OMP T	OMP S	OMP E	Numba T	Numba S	Numba E	Mejor
1	796.495	0.98	0.98	696.564	1.12	1.12	Numba
2	468.221	1.67	0.84	452.235	1.73	0.86	Numba
4	279.796	2.79	0.70	284.197	2.75	0.69	OpenMP
8	182.725	4.28	0.53	185.490	4.22	0.53	OpenMP

Resumen a 8 threads

Escena	Config.	Ts	OMP T8	Numba T8	Mejor
scene.txt	small	4.075	0.765	0.659	Numba
scene.txt	medium	64.690	15.464	12.602	Numba
scene.txt	large	533.884	112.857	102.854	Numba
scene_many.txt	small	6.177	1.251	1.298	OpenMP
scene_many.txt	medium	98.010	22.855	23.217	OpenMP
scene_many.txt	large	782.014	182.725	185.490	OpenMP

Escena	Config.	OMP S8	OMP E8	Numba S8	Numba E8
scene.txt	small	5.32	0.67	6.18	0.77
scene.txt	medium	4.18	0.52	5.13	0.64
scene.txt	large	4.73	0.59	5.19	0.65
scene_many.txt	small	4.94	0.62	4.76	0.59
scene_many.txt	medium	4.29	0.54	4.22	0.53
scene_many.txt	large	4.28	0.53	4.22	0.53

Numba fue mas rapido que OpenMP en 19 de 24 puntos comparables. En promedio, el cociente `T_OpenMP/T_Numba` fue 1.13x. La mayor ventaja de Numba aparecio en scene.txt `large` con p=1, donde OpenMP demoro 1.45x el tiempo de Numba. La menor razon fue 0.96x en scene_many.txt `small` con p=8, donde OpenMP fue levemente mas rapido.

La eficiencia cae al aumentar p. Con 8 threads, OpenMP alcanza speedups entre 4.18x y 5.32x, con eficiencias entre 0.52 y 0.67; Numba alcanza speedups entre 4.22x y 6.18x, con eficiencias entre 0.53 y 0.77. La caida se explica por overhead de creacion/sincronizacion de trabajo, barreras implicitas, planificacion, presion de cache y saturacion de recursos. Ademas, el equipo tiene 6 cores fisicos y 12 threads logicos, por lo que al usar 8 threads ya se empieza a depender parcialmente de SMT y de recursos compartidos del procesador.

Graficos

Los graficos fueron generados desde el mismo `results.csv`.

Scheduling

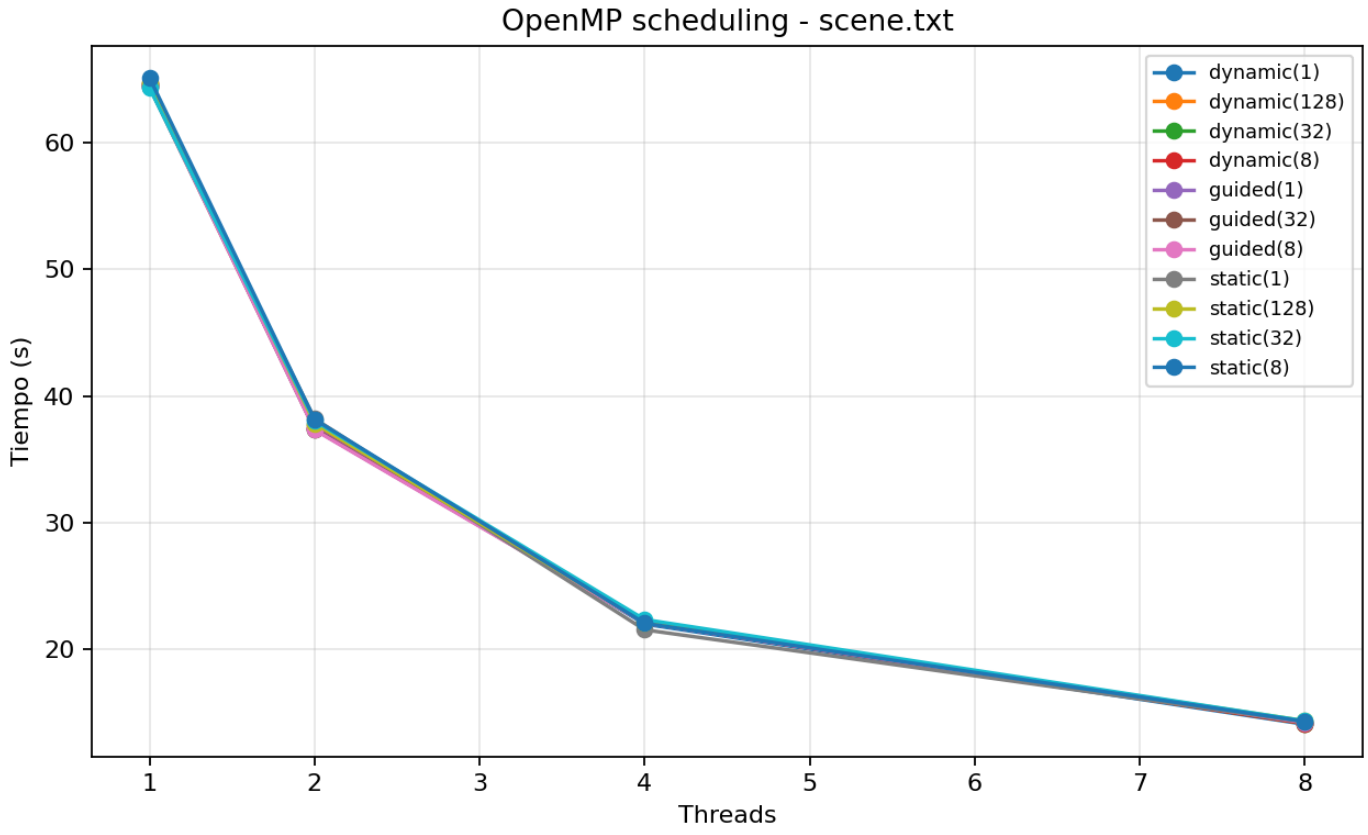

Figure 1: Scheduling scene.txt

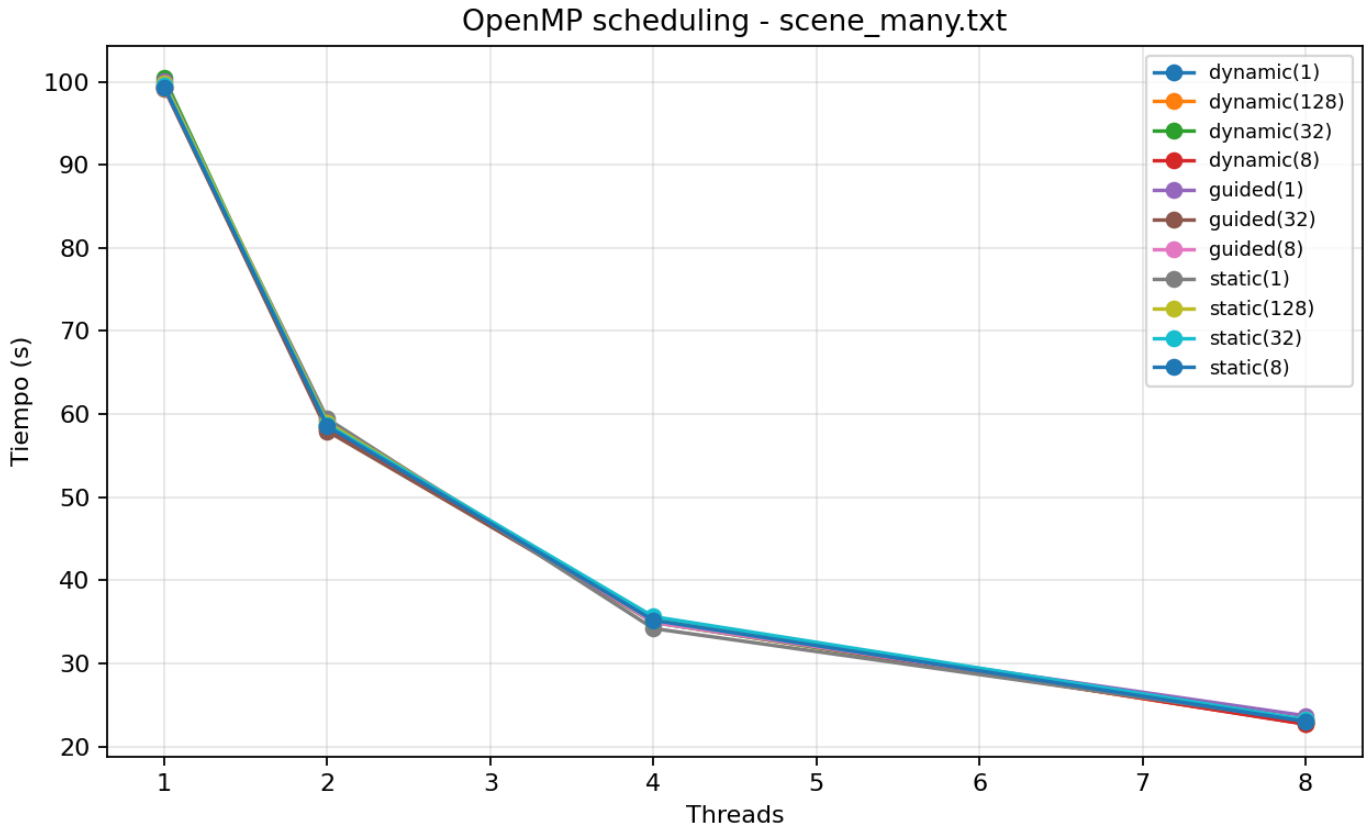

Figure 2: Scheduling scene_many.txt

scene.txt small

scene.txt medium

scene.txt large

scene_many.txt small

scene_many.txt medium

scene_many.txt large

Segundo computador

La pauta pide repetir los experimentos en un segundo computador y comparar numero de cores fisicos, threads logicos, frecuencia y resultados. En esta carpeta solo existen resultados del computador local (`results/full_local_`). Por lo tanto, esta seccion queda pendiente hasta copiar una corrida `--full` de otra maquina.

Cuando exista esa corrida, se debe agregar una tabla con el hardware del segundo equipo y comparar al menos: tiempos seriales, speedup a 8 threads, eficiencia, y diferencias atribuibles a cantidad de cores, frecuencia, cache, SMT y carga del sistema.

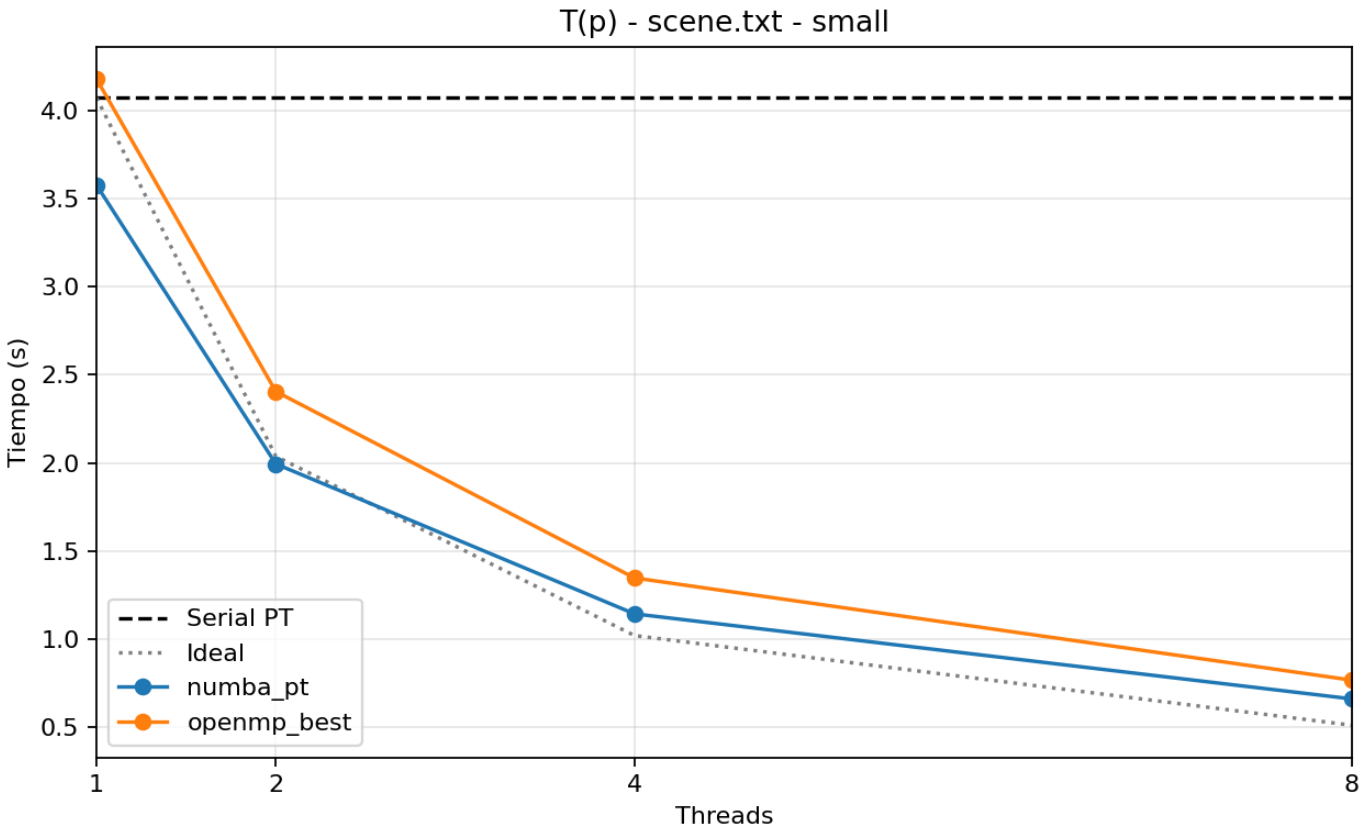

Figure 3: Tiempo scene.txt small

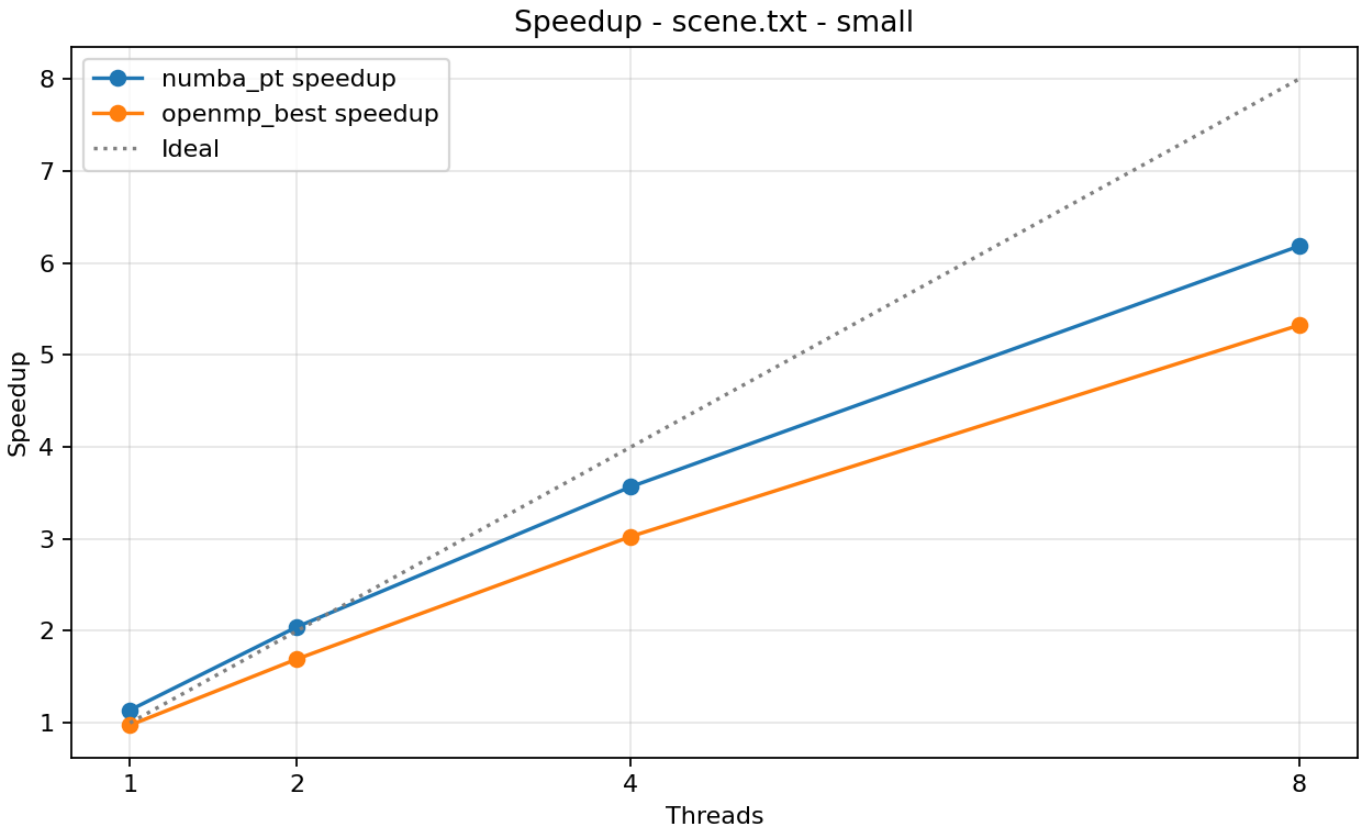

Figure 4: Speedup scene.txt small

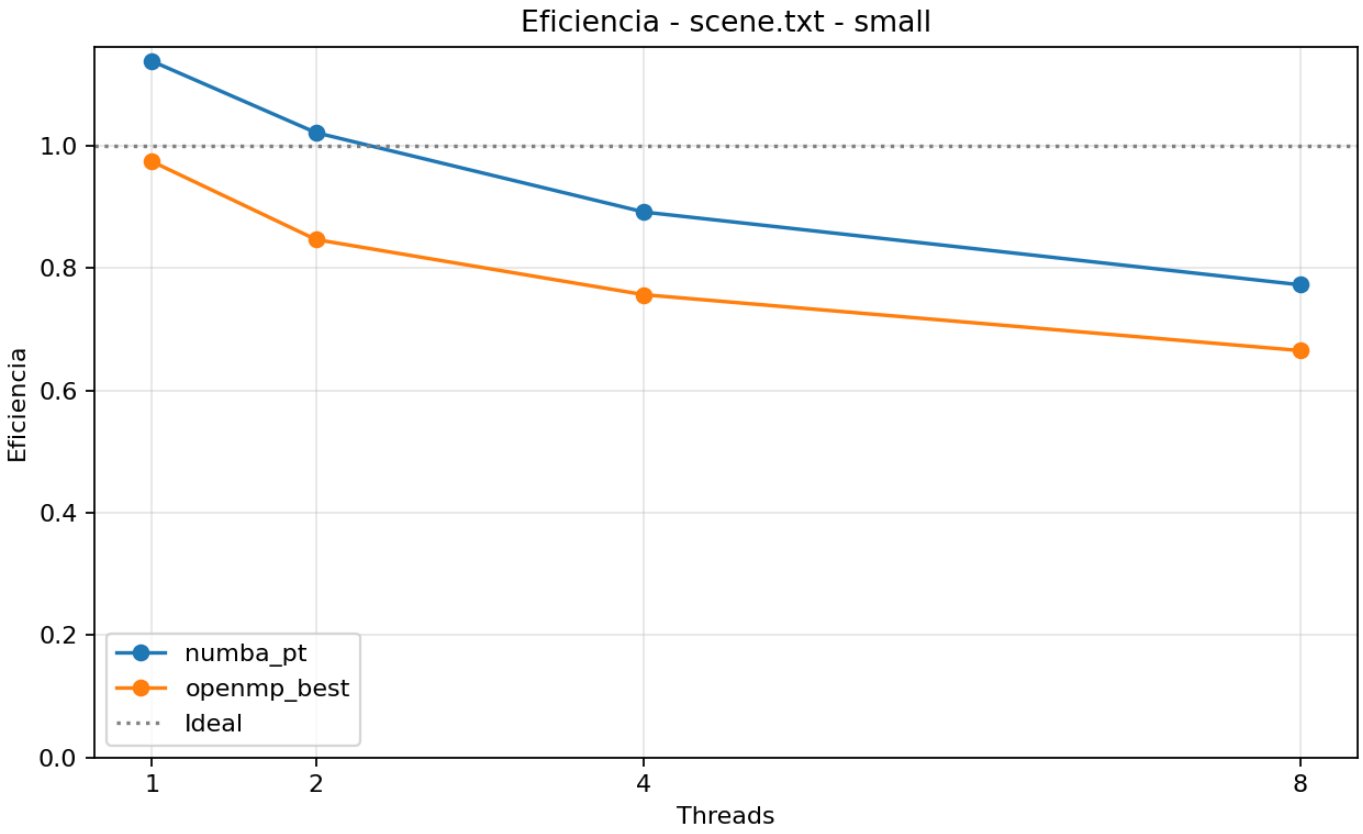

Figure 5: Eficiencia scene.txt small

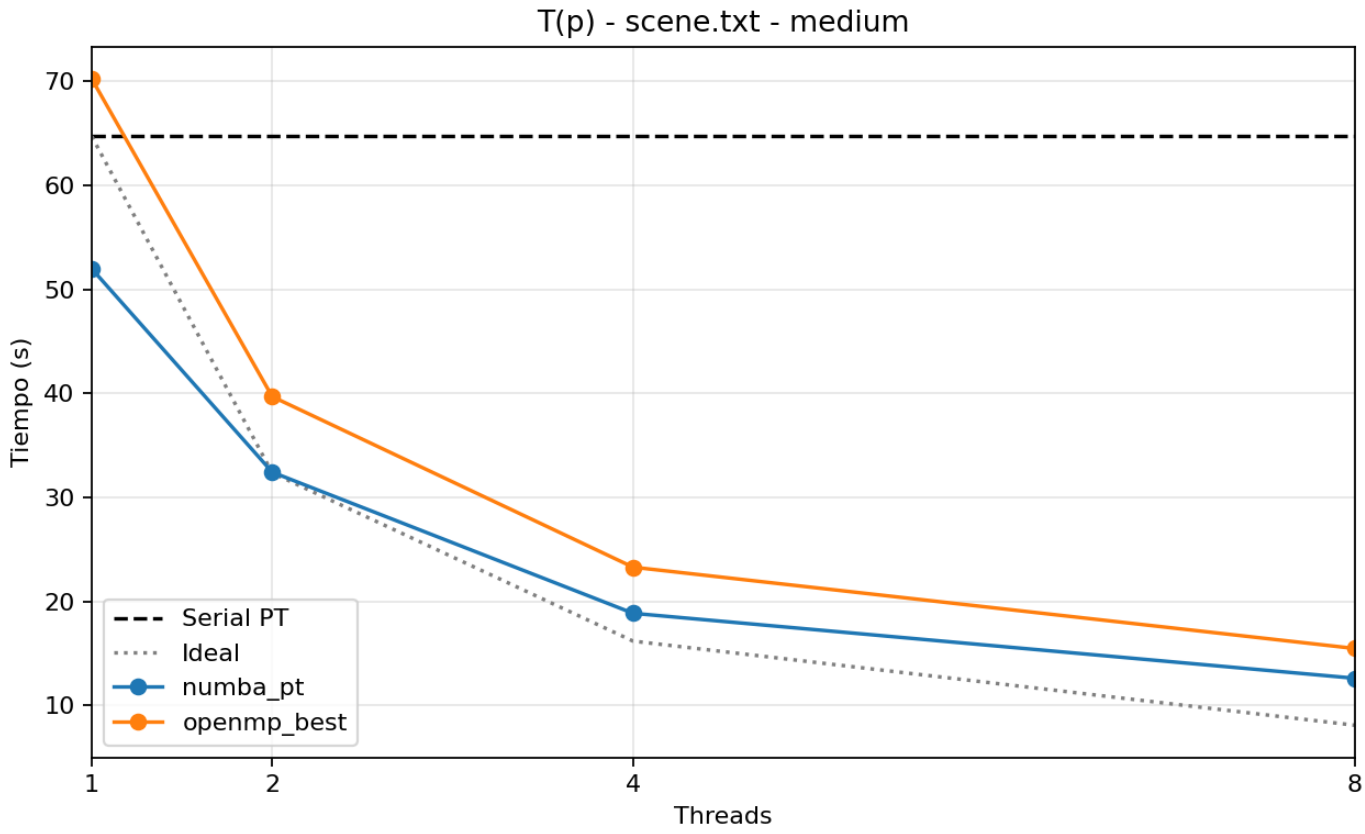

Figure 6: Tiempo scene.txt medium

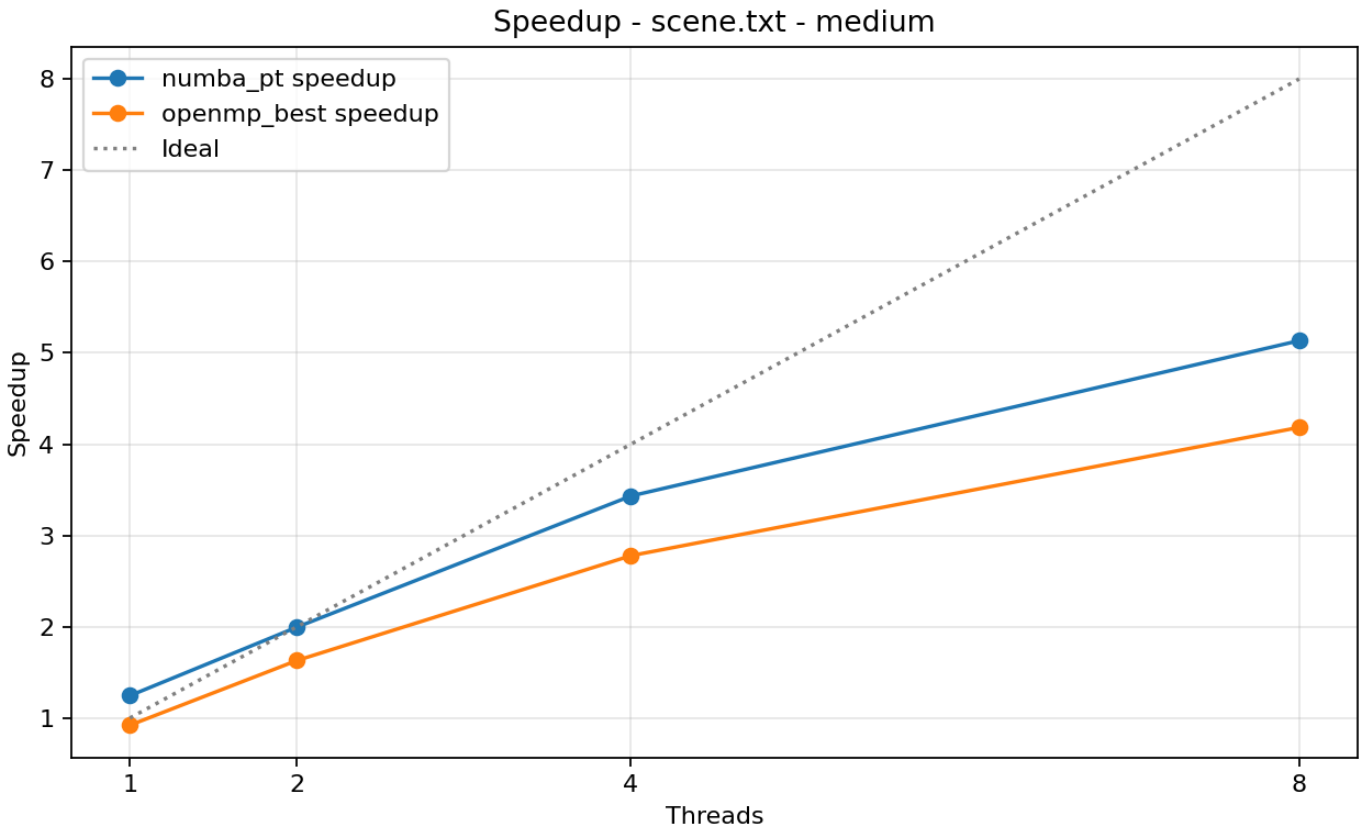

Figure 7: Speedup scene.txt medium

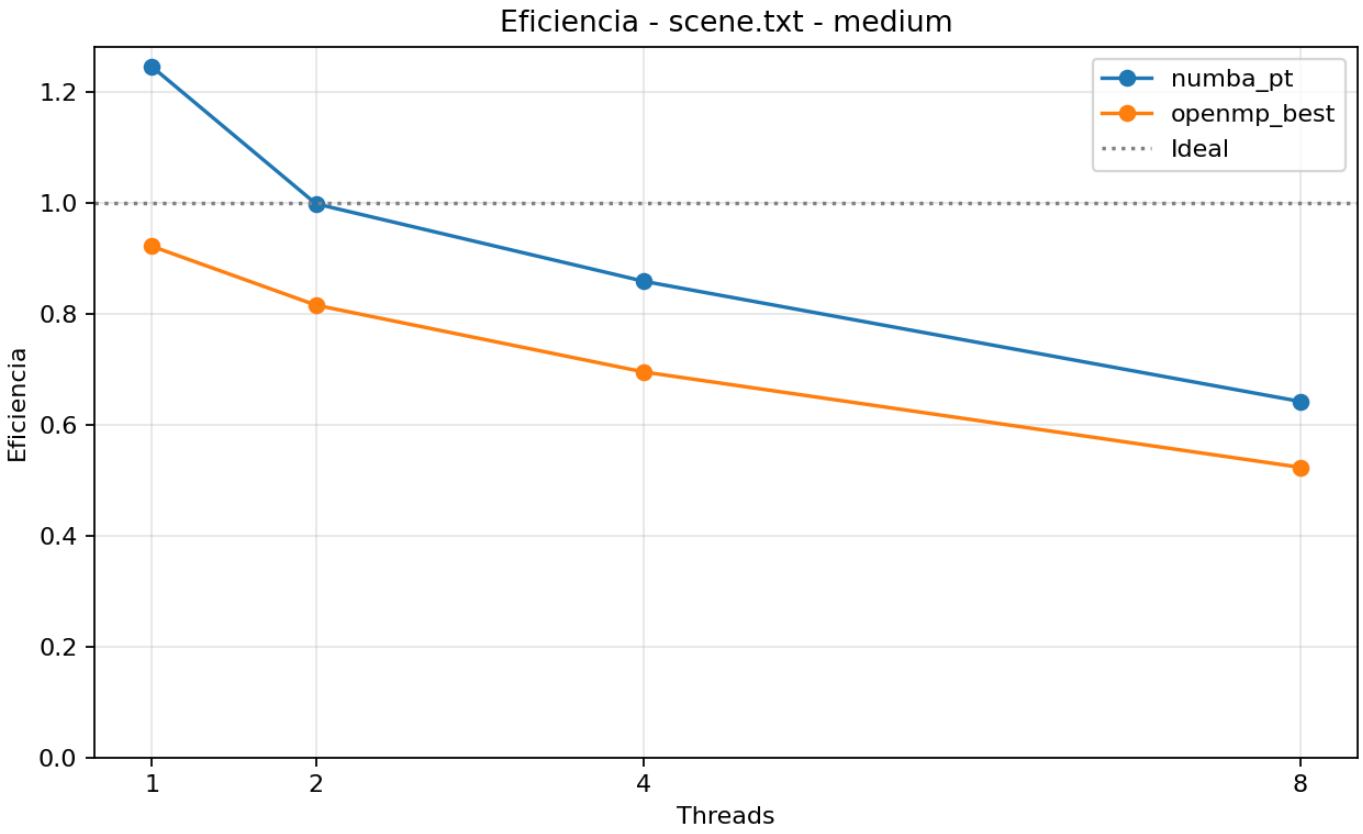

Figure 8: Eficiencia scene.txt medium

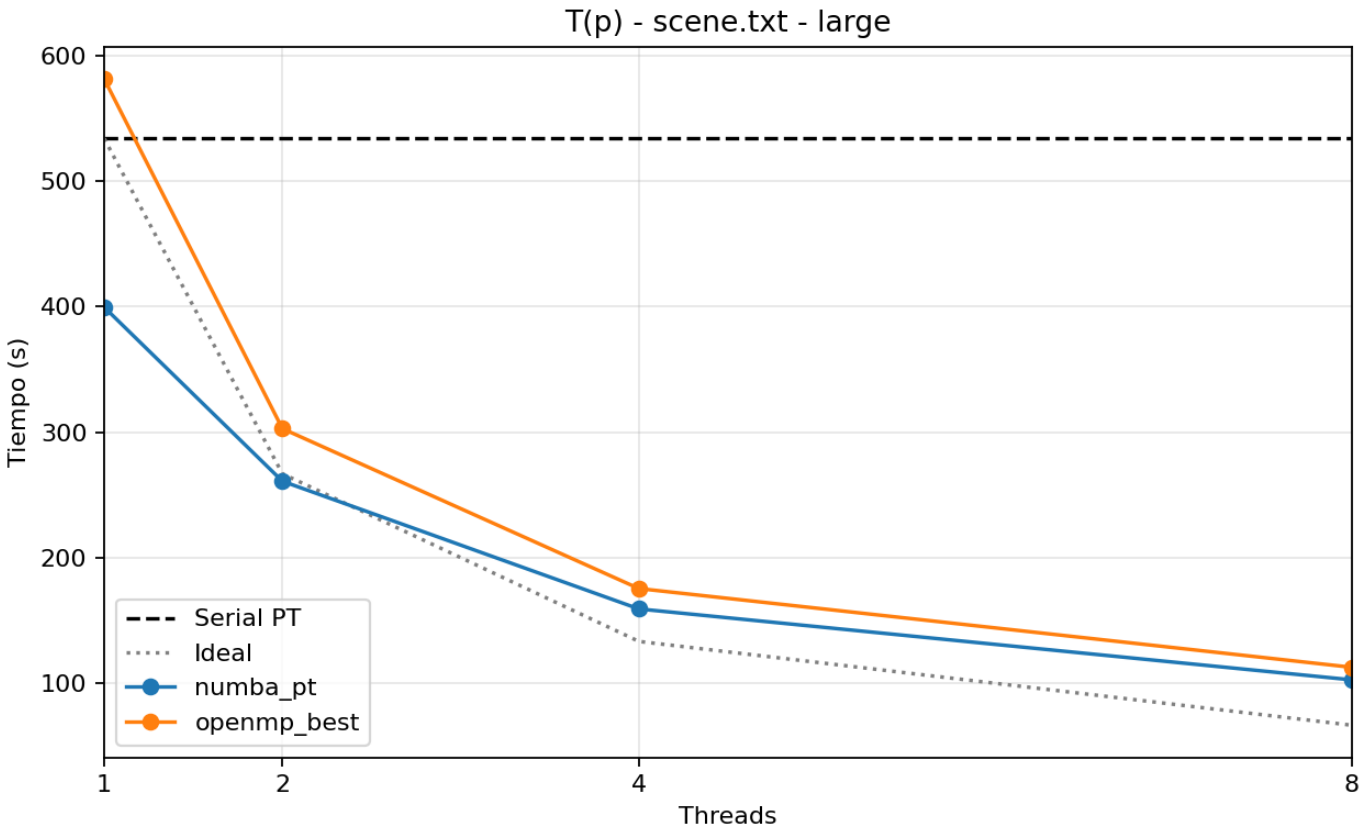

Figure 9: Tiempo scene.txt large

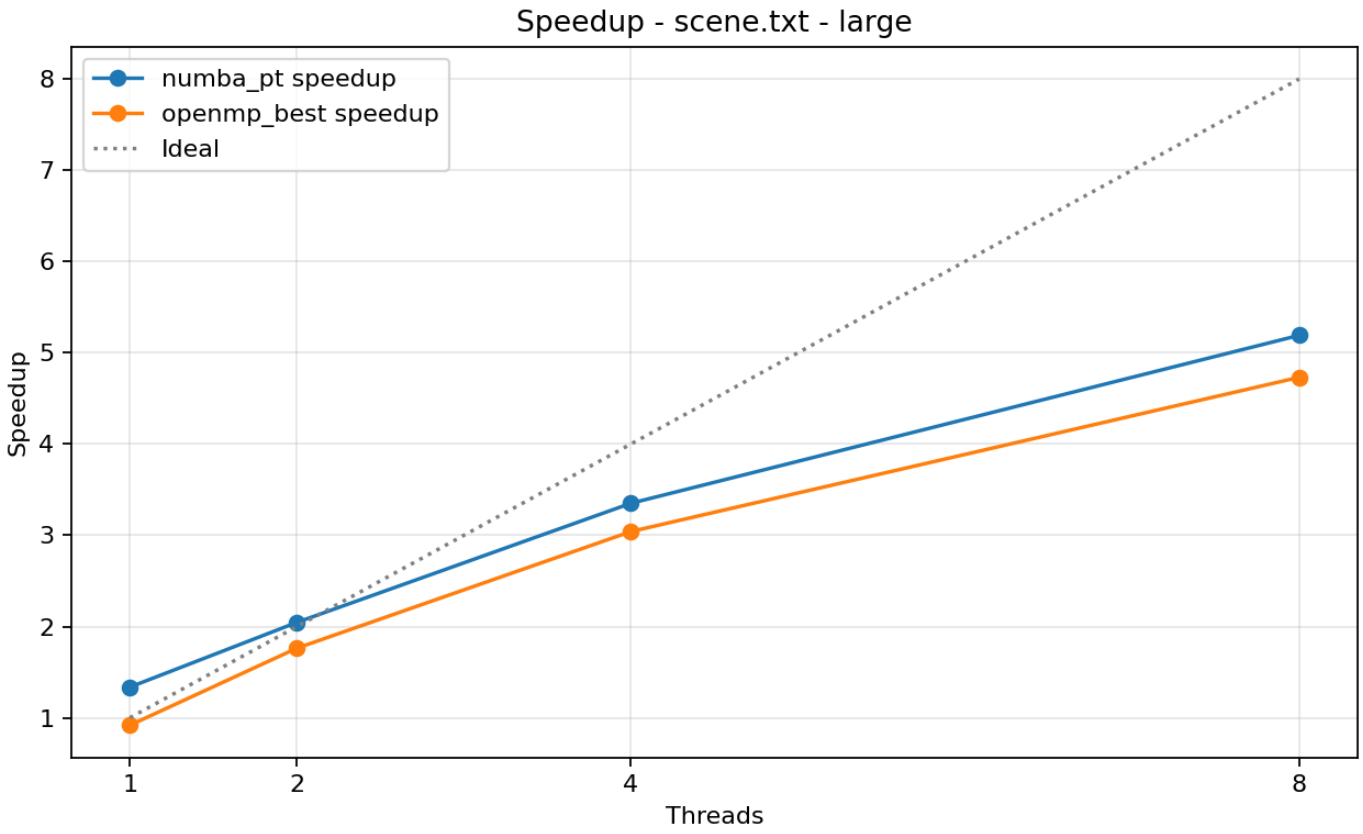

Figure 10: Speedup scene.txt large

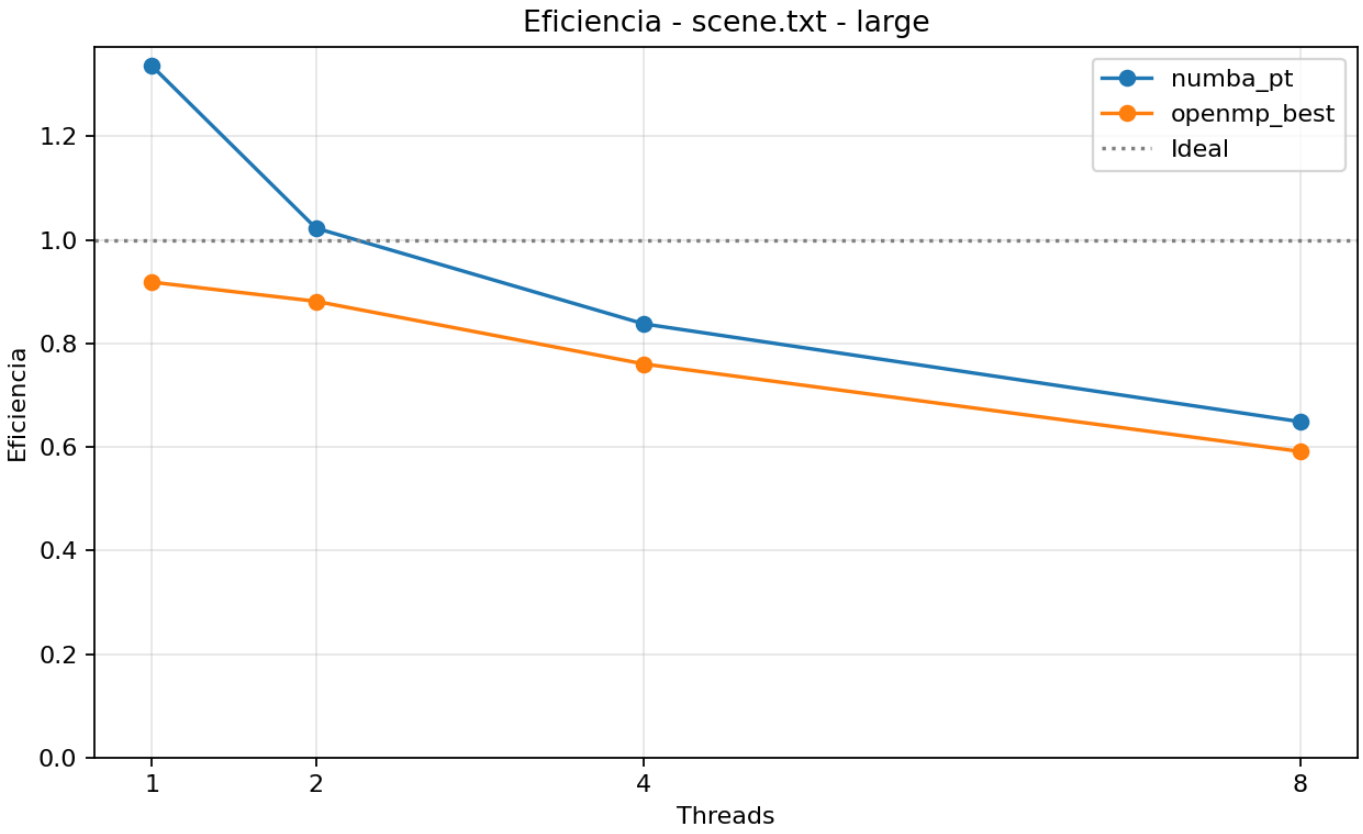

Figure 11: Eficiencia scene.txt large

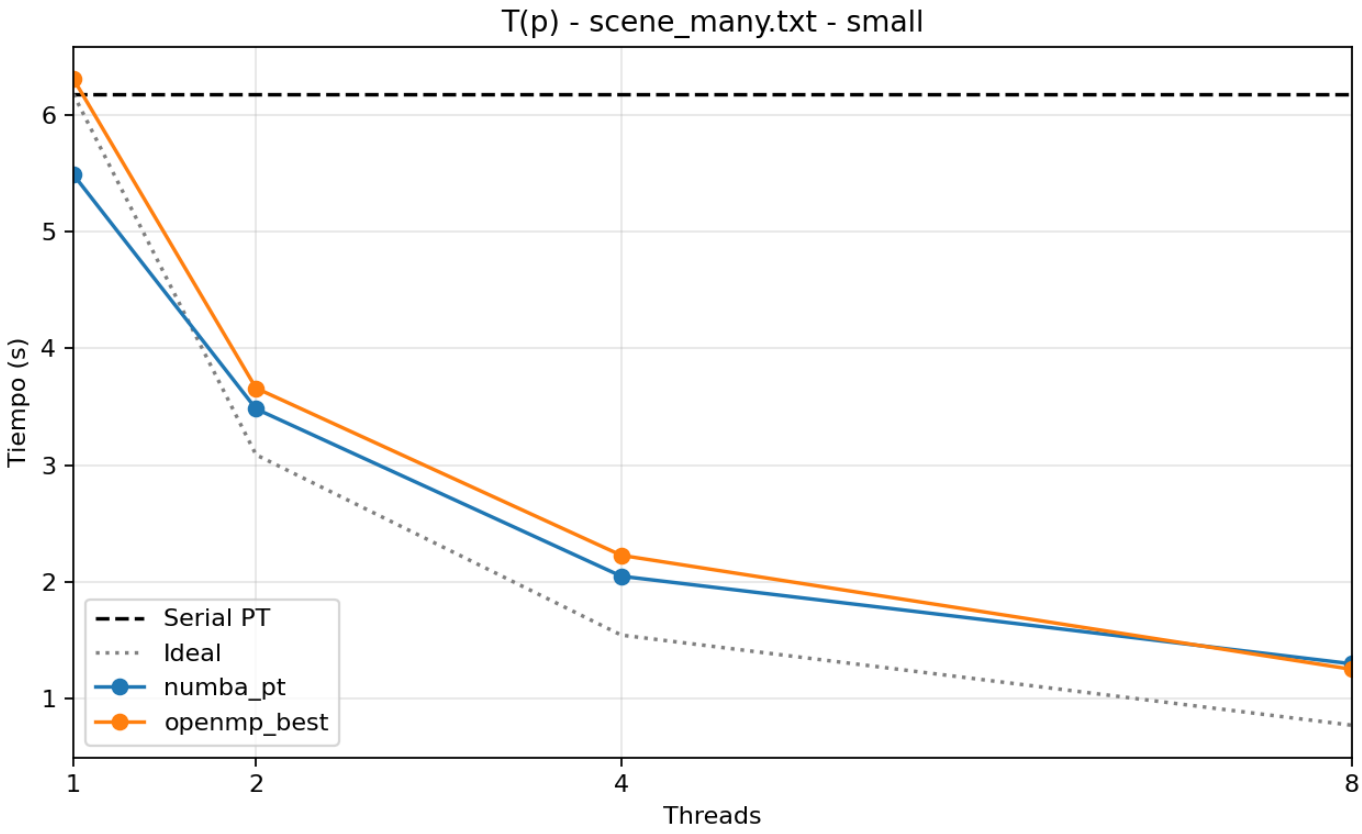

Figure 12: Tiempo scene_many.txt small

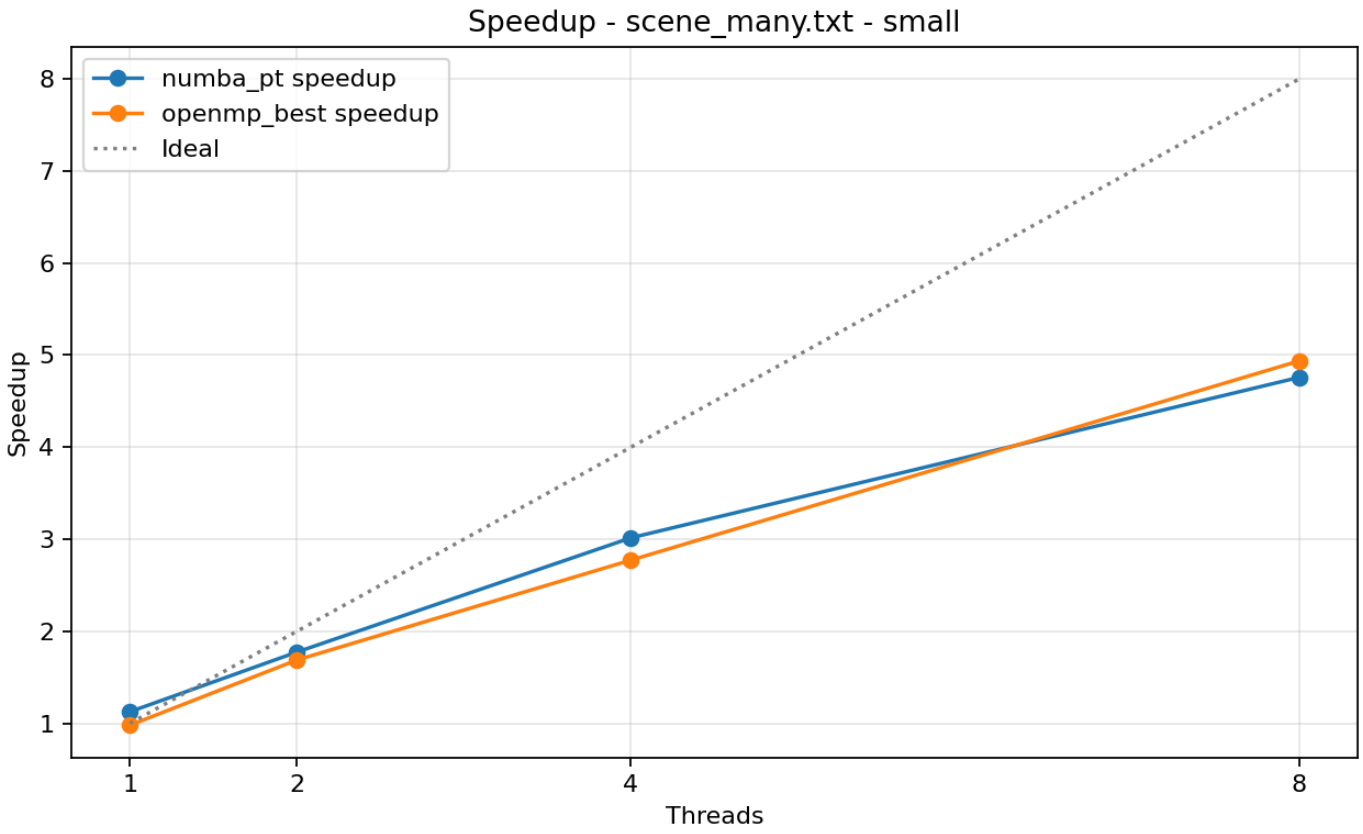

Figure 13: Speedup scene_many.txt small

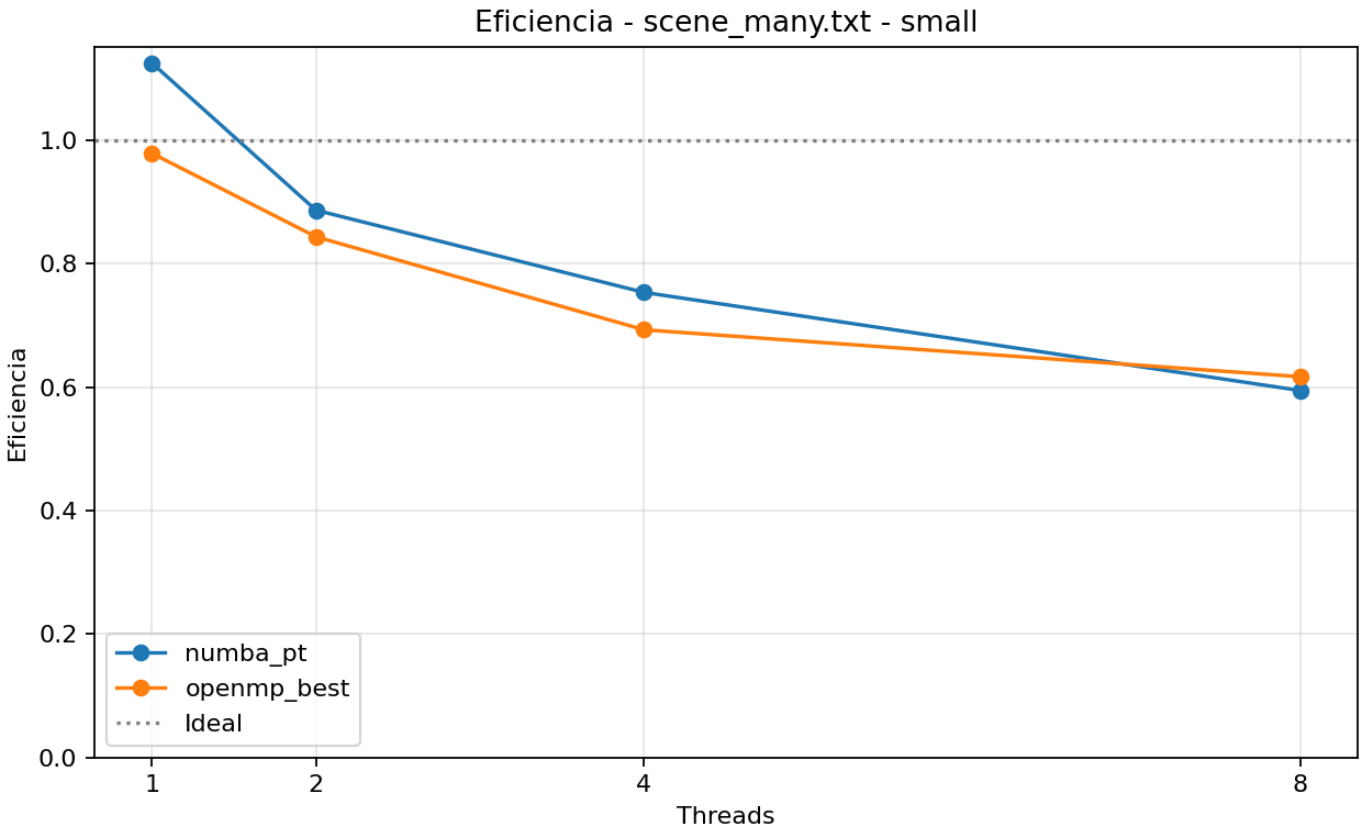

Figure 14: Eficiencia scene_many.txt small

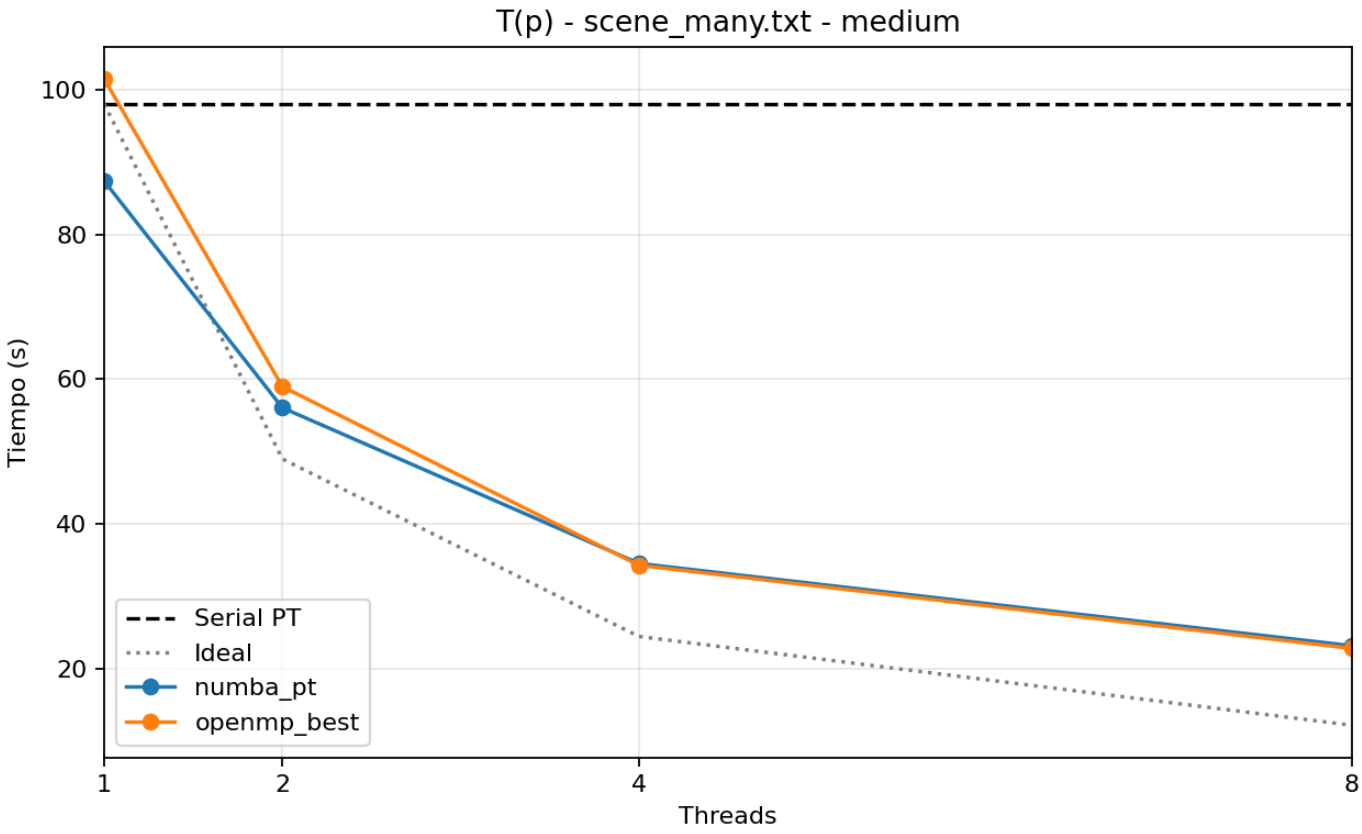

Figure 15: Tiempo scene_many.txt medium

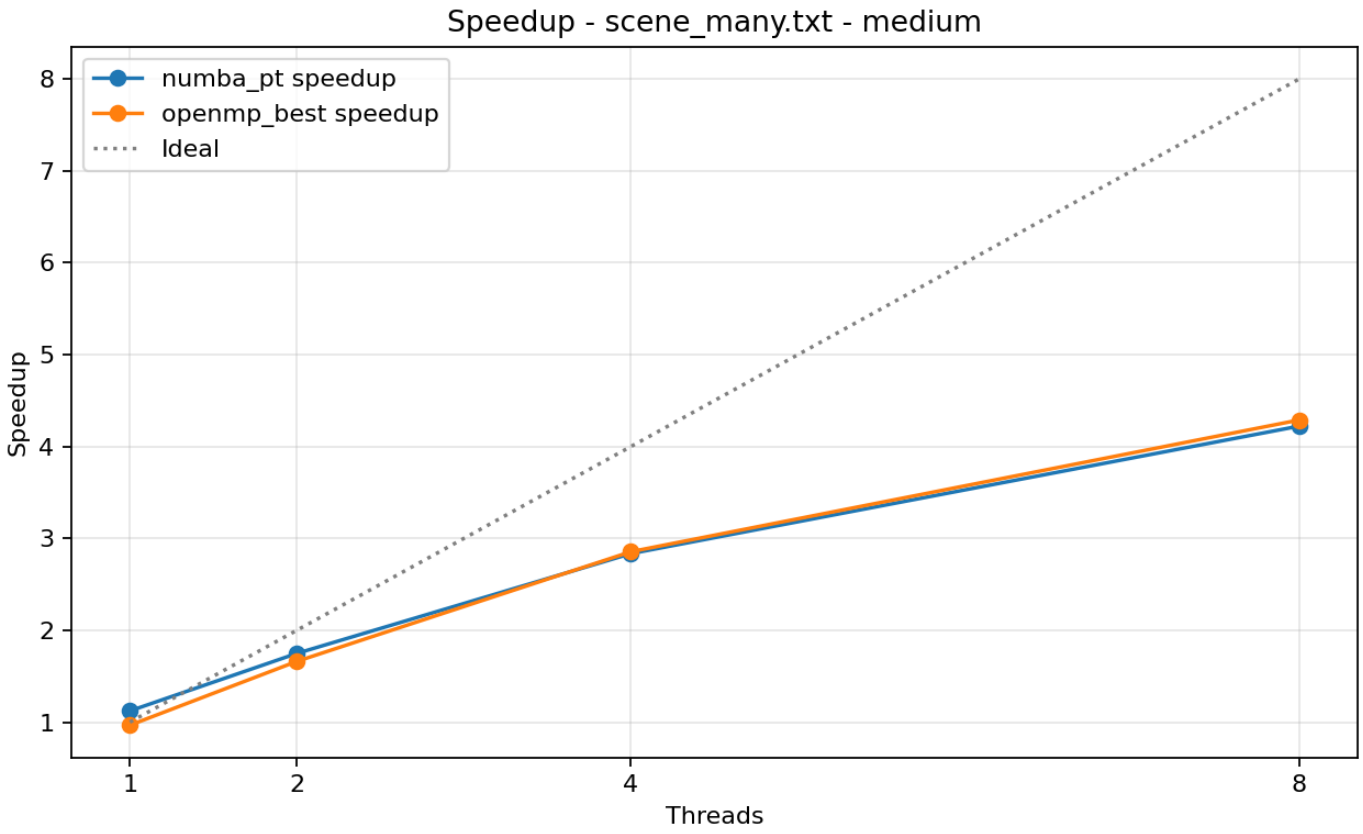

Figure 16: Speedup scene_many.txt medium

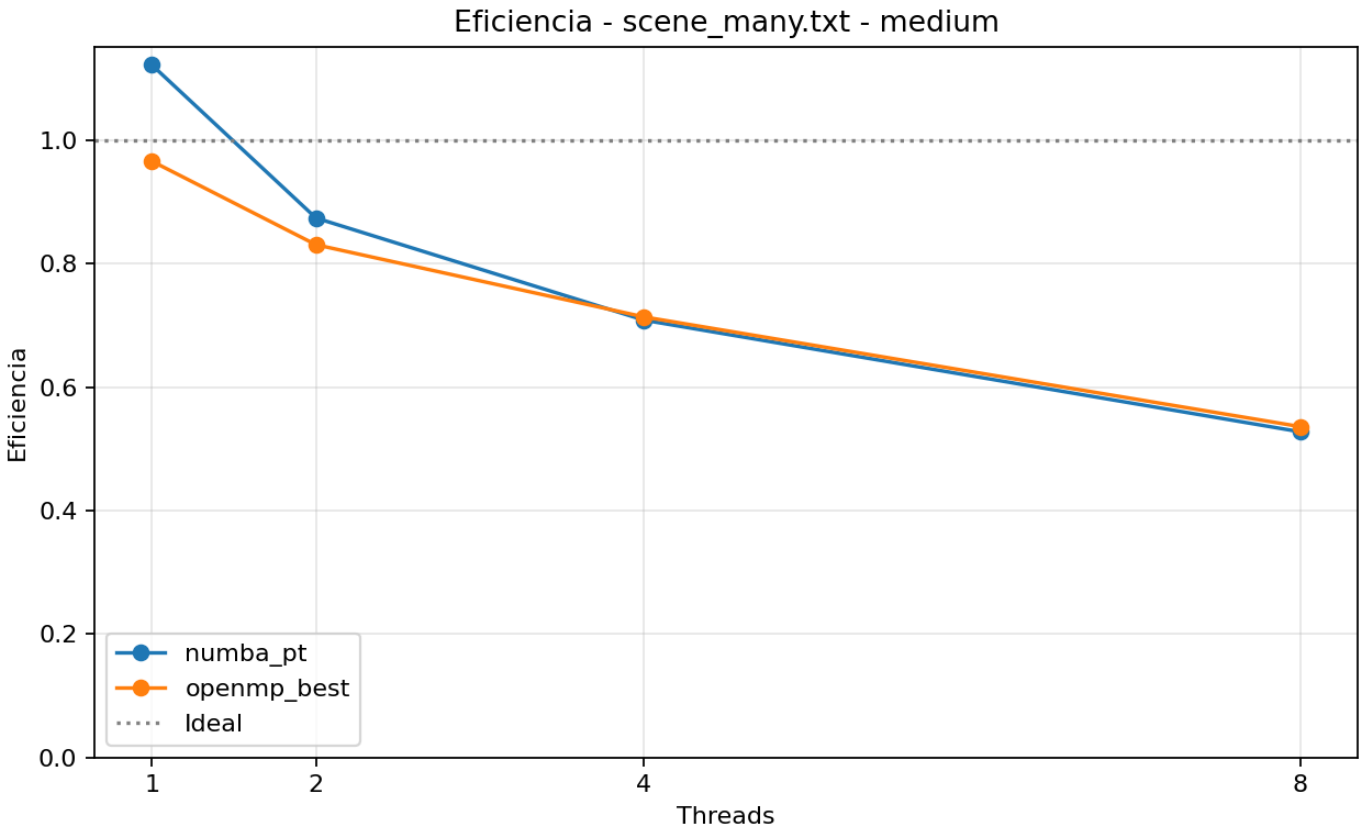

Figure 17: Eficiencia scene_many.txt medium

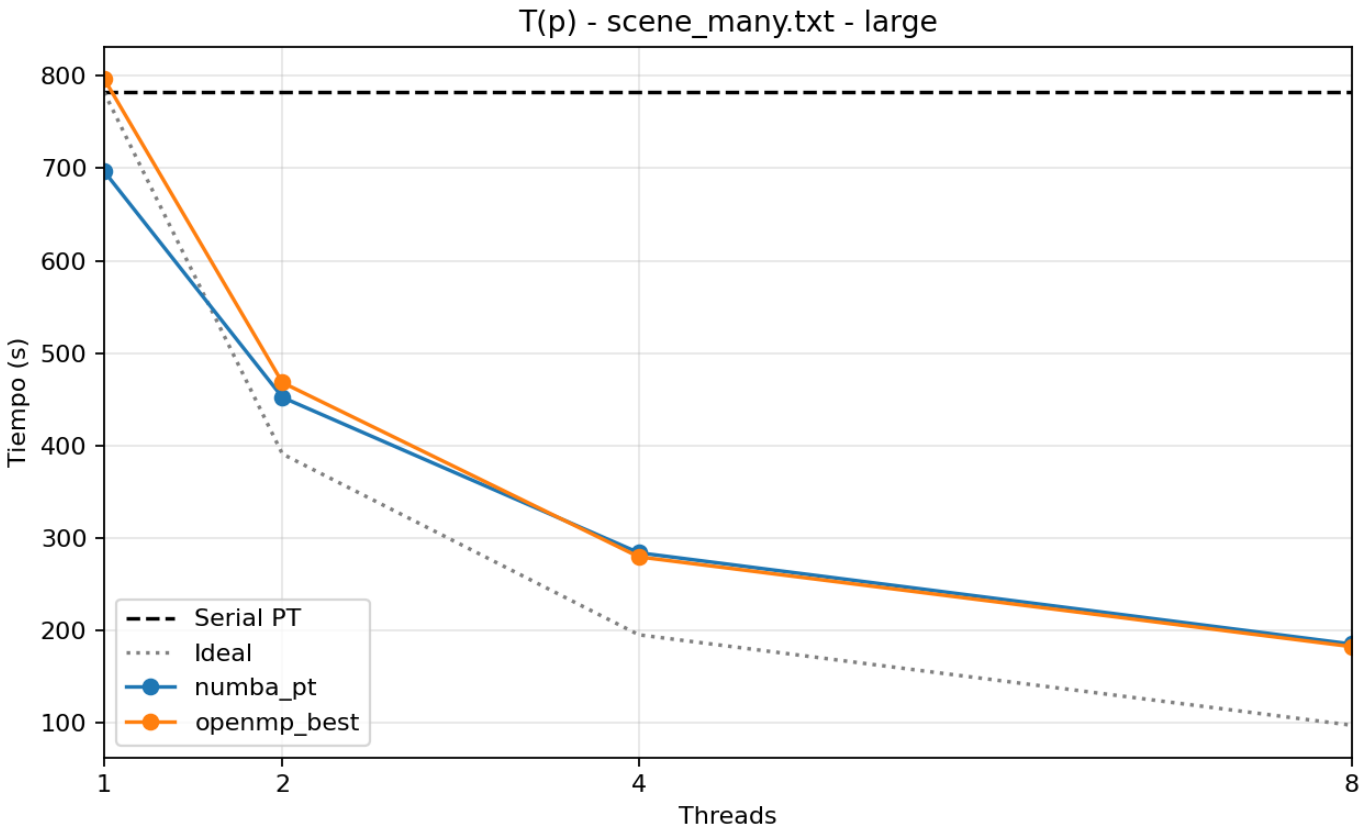

Figure 18: Tiempo scene_many.txt large

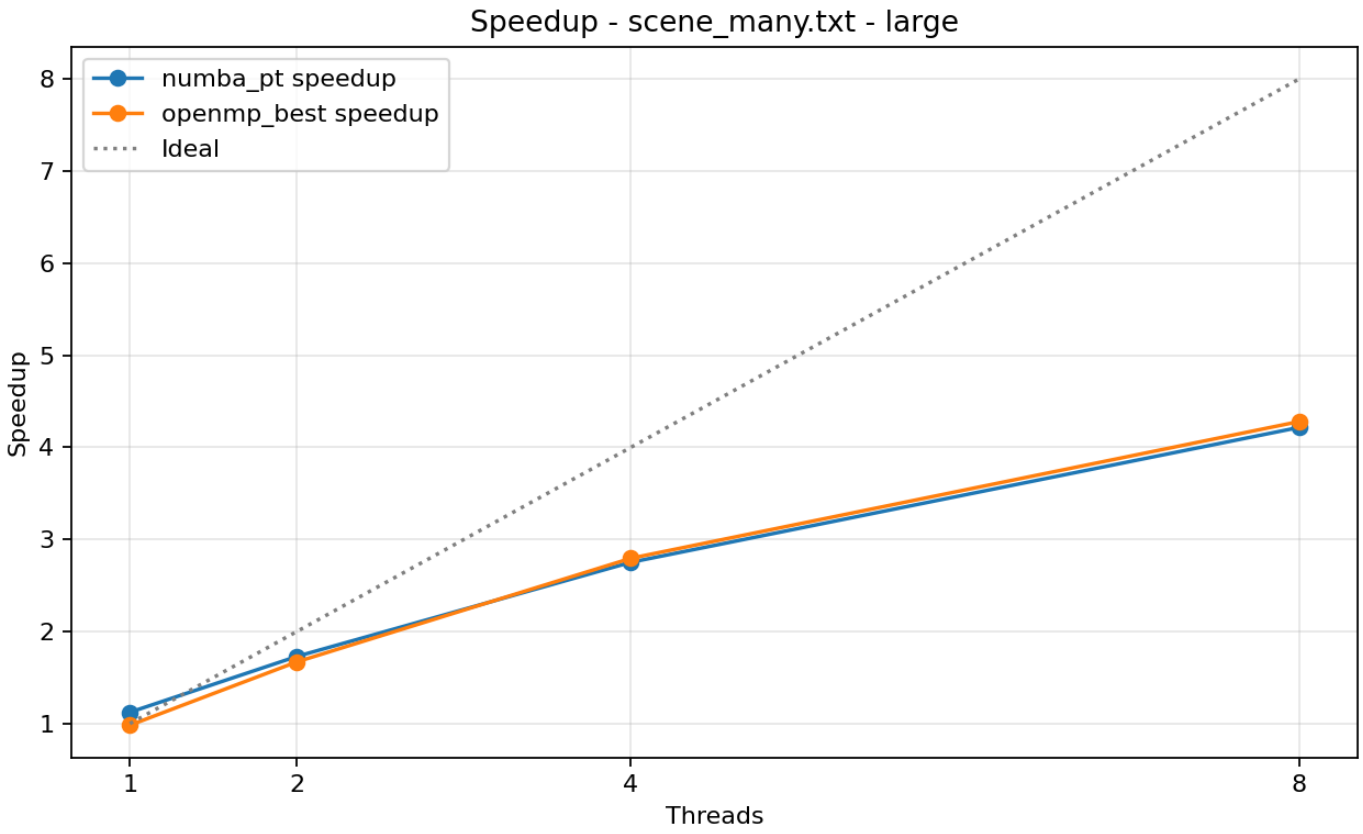

Figure 19: Speedup scene_many.txt large

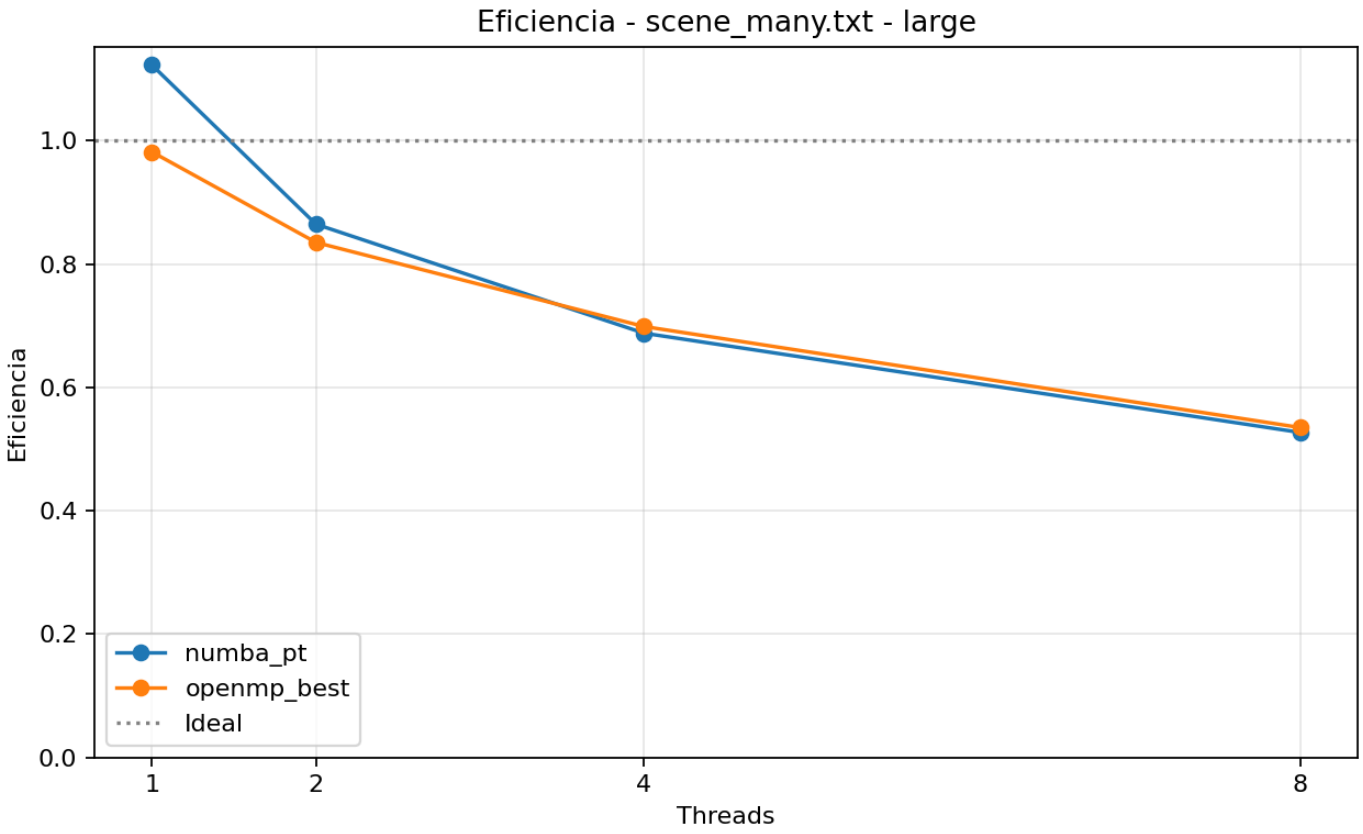

Figure 20: Eficiencia scene_many.txt large